CSC 4101 – HW2

John Holcomb

1.

<id-list> -> <id-list> COMMA ID would cause infinite recursion. <decl-list> -> <decl> SEM <decl-list> | <decl> both have common prefixes, which would not parse correctly with only one token lookahead.

Correct grammar:

<var-decl> -> VAR <decl-list>

<decl-list> -> <decl> <decl-tail>

<decl-tail> -> SEM <decl-list> | ε

<decl> -> <id-list> COLON ID

<id-list> -> ID <id-list-tail>

<id-list-tail> -> COMMA ID <id-list-tail> | ε

2.

Left:

S => S S *

=> S S + S *

=> a S + S *

=> a a + S *

=> a a + a *

Right:

S => S S *

=> S a *

=> S S + a *

=> S a + a *

=> a a + a *

3.

Example:

if e then statement1

if e then statement1 else statement2

if e then begin statement1; statement2 end

if e then begin statement1; statement2 end else begin statement3; statement4 end

The 1960 syntax created ambiguous grammar for dangling elses. The grammar could allow the following two valid derivations from “if e1 then if e2 then S1 else S2”:

if e1 then (if e2 then S1 else S2)

if e1 then (if e2 then S1) else S2

The revised syntax made that impossible by adding the unconditional statement block, which properly nests if/elses using the begin and end terms.

4.

<statement> -> <if-statement>

| <assignment>

| ...

<if-statement> -> IF <expression> THEN <statement-list>

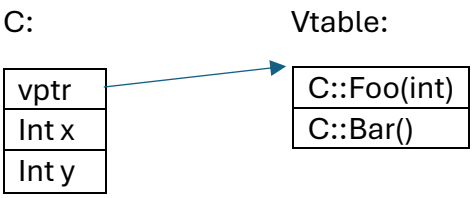
{ ELSIF <expression> THEN <statement-list> }

[ELSE <statement-list>]

END IF

<statement-list> -> <statement> { ; <statement> }

5.



A new C object is created with the location stored in p, with x = 3 and y = 5. The vptr within object p is loaded and indexes vtable[0] to get foo(int). Foo passes the argument 37, which sets this->x to 37, changing x to 37 within the object, and returns the “this” pointer. The vptr in the object from the this pointer is loaded to call vtable[1] to get bar(). Bar computes this->x + this-> y, which returns 42. I is set to 42.